

Doppelgänger — Full Game Design & Godot Build Plan

Source: hand-written jam notes ("Doppelgängen") + one new requirement added on top: **doppelgängers can also collect enemy drops**. Engine target: **Godot 4.x (GDScript)**. If your team is on Godot 3.x, translate `@export / @onready` back to `export / onready`, and note that `move_and_slide()` takes no arguments in Godot 4 (velocity is read from the `velocity` property instead).

0. TL;DR

A top-down, **not endless**, round-based action game. **8–10 rounds**, then a final climax. Every round you record yourself. At the start of the next round, that recording comes back to life as a **doppelgänger** — an AI ghost that walks your old path, but will break off to fight you, dodge you, and dodge the exact trap that killed it. Every round adds one more of them. You can eventually **possess a doppelgänger**, walk among your own clones undetected, and **sabotage** them from within — at the risk of your own hidden body being found.

1. Source Notes Recap (what was on the page)

- Top-down, Godot, not endless → **8–10 rounds before a final climax**.
- Player plays and dies / completes the first round → **starting round 2**, there is **1 doppelgänger + 1 player**. The doppelgänger count then **increases by 1 every round**.
- Top-down game → hazards placed in the arena: **pits, spikes, fire, etc.**
- **Player**: integer health bar, **3 hits OR 1 hazard = death**. Can shoot from the start by default. Can take control of a clone (from a special pickup, holdable for later use) — while possessing, you lose view/control of your real body (which becomes vulnerable), and can switch back anytime. Skills unlock permanently via XP from killing doppelgängers/enemies; temporary skills come from drops.
- **Enemy**: spawns randomly, chases if player is within an aggro radius, contact deals 1 HP damage, can drop coins/health packs.
- **Doppelgängers**: count grows by 1 each round/death. Replay the player's original movement, **but** have their own will — attack as simple AI if the player is in range, dodge player attacks, and avoid the trap that killed them. They inherit the player's **permanent** skills as of the round they were born in. A **special elite doppelgänger** (random, one at a time, every ~2 minutes) drops the clone-control item, has higher HP, and all skills unlocked. Possessing a clone lets the player **mix into** the doppelgänger group and **sabotage** them — this alerts the others, who can hunt down the real body, and can kill the possessed clone if control isn't released in time. You can sabotage as many as you want, at the risk of your own life.

- **Player (stealth):** bushes let the hidden real body hide while you're off possessing a clone — but that hideout **stops working once you sabotage** from it.
 - **New requirement (added by team):** doppelgängers should also be able to **walk over and collect enemy drops** (coins/health packs), not just the player.
-

2. Design Assumptions & Resolved Ambiguities

The notes are a scratch outline, not a spec — here's how this plan resolves the fuzzy parts. Flag any of these in standup if you disagree; they're all one-line code changes to flip.

1. **Round 1 has zero doppelgängers.** Both branches in the notes ("if died → round 1 → 1 doppel + 1 player" / "else → round 2 → 1 doppel + 1 player") land on the same place: whatever happens in round 1, **round 2 is the first round with a doppelgänger**. So: `doppelganger_count(round N) = N - 1`. Simple, monotonic, matches "increases the count by 1 each round."
 2. **Hazards are instant-kill and separate from the 3-hit HP pool.** "3 hits or 1 hazard" = two different death conditions, not hazards costing 1 HP.
 3. **Clone-control is an inventory item**, not an instant-use ability — picked up, held, activated on demand (per "can hold the pickup for later use").
 4. **Only permanent skills get baked into a doppelgänger's skill snapshot.** Temp skills (from drops) are short-lived buffs for the live player only — they don't get inherited, because the notes only say doppelgängers "gain the permanent skills of players."
 5. **"Sabotage"** = while possessing doppelgänger A, walk up to doppelgänger B and destroy/disable it. This is the mechanic that raises alert and can get your clone killed.
 6. **Bush-hiding only matters for your real (unpossessed) body** while you're off in a clone — that's the scenario the notes describe it in.
 7. **Climax structure is a decision, not a spec** — Section 15 gives two concrete options; pick one early since it affects how much extra content you need to build.
 8. **New: doppelgängers actively route to and pick up nearby drops** during their replay state (coins get "denied" to the player economy, health packs heal the doppelgänger) — see Section 4.7 and the `Doppelganger._do_collect()` code.
-

3. Core Gameplay Loop

```
Round N starts
→ spawn (N-1) doppelgängers from stored recordings
→ spawn enemies, hazards, pickups, bushes
→ player plays live, gets recorded every physics frame

Round ends when: player dies OR player completes round
→ finalize this round's Recording (path + actions +
  which hazard killed them, if any + current perm skills)
→ store it in GameManager.recordings[]

If round N == last normal round → go to Climax Arena
Else → round N+1 starts (loop)
```

Each stored `RoundRecording` becomes exactly one doppelgänger, forever, for the rest of the run. By round 9 you're sharing the arena with 8 ghosts of your past selves.

4. System Design

4.1 Player

- Top-down `CharacterBody2D`. Default weapon: ranged shoot, unlocked from the start.
- HP pool (default 3), `take_hit(n)` for combat damage, `die(hazard_id)` for instant hazard death (bypasses HP).
- Records its own transform every physics tick via a `RecordingComponent`.
- Holds at most one clone-control item at a time; activating it possesses the nearest doppelgänger in range.

4.2 Enemies

- Simple `CharacterBody2D` "mobs." Idle until player enters `aggro_radius`, then seek toward the player. Contact = 1 HP damage to whoever it touches (player **or** doppelgänger — see 4.7). Drops loot on death.

4.3 Hazards

- Static `Area2D` triggers: pit / spikes / fire. Instant death to the player on contact. Each hazard instance has a unique `hazard_id` string, which is how a doppelgänger "remembers" the specific hazard that killed its original run.

4.4 Doppelgängers

State machine per doppelgänger:

```

REPLAY —(player in detection radius)→ COMBAT
REPLAY —(nearby unclaimed drop)→ COLLECT → REPLAY
REPLAY —(recording runs out)→ ALERT_SEARCH
COMBAT —(player leaves range)→ REPLAY
COMBAT —(incoming bullet)→ DODGE → COMBAT
(any) —(possessed by real player)→ POSSESSED
(any) —(sabotage broadcast heard)→ ALERT_SEARCH

```

- **REPLAY**: walks its own recorded position history, frame by frame, and replays logged actions (e.g. shoot events) at the matching frame.
- **COMBAT**: seeks the player, fires using its *own* skill snapshot (damage/fire-rate from whatever round it was born in), steers away from the specific hazard tile it died to in its own recording.
- **ALERT_SEARCH**: roams/hunts for the player's last-known position; entered when another doppelgänger is sabotaged (see 4.6), or when a recording naturally ends.
- **COLLECT**: new behavior — breaks off briefly to grab a nearby coin/health drop, then resumes REPLAY.
- **POSSESSED**: driven directly by player input while the player has taken control of it.

Every 2 minutes, one random living doppelgänger is upgraded into a **special/elite**: higher HP, all skills unlocked. Killing it drops the clone-control pickup.

4.5 Clone Control & Possession

- Picked up from a defeated elite doppelgänger; held until used.
- Activating it: player's real body goes idle (still physically present, still able to take damage, camera and input move to the doppelgänger).
- While possessed, that doppelgänger is *not* attacked/detected by the others (you're "blending in") — until you sabotage.
- Release control anytime to snap back to your real body.

4.6 Sabotage, Alert & Bushes

- While possessing doppelgänger A, get close to doppelgänger B and trigger sabotage → B is destroyed, and **all other doppelgängers** switch to **ALERT_SEARCH** (they now actively hunt for the real player body).
- If a possessed clone dies (killed by other alerted doppelgängers) while you're still inside it and haven't released control, you get ejected and take a damage penalty.
- Bushes hide the real (idle) body from detection while it's inside a bush footprint — but the specific bush you were hiding in becomes **compromised** (stops hiding you) the moment you sabotage anything from that session. No hard cap on how many times you can sabotage — the growing alert state is the risk.

4.7 Pickups & Economy

- **Coin**: currency, player-only value.
- **Health pack**: heals whoever picks it up — including doppelgängers.

- **Clone-control pickup:** dropped only by the elite doppelgänger.
- **Temp-skill drop:** short random buff (fire rate, move speed, etc.), player-only, does not persist into doppelgänger snapshots.
- **NEW** **Doppelgängers can walk over and collect nearby drops themselves** (`COLLECT` state) — a health pack heals them, making late-round doppelgängers tougher if you leave loot lying around; a coin picked up by a doppelgänger is simply denied to the player. This gives players a reason to race doppelgängers to loot.

4.8 Skills (Permanent + Temporary)

- XP from killing enemies/doppelgängers → permanent skill points (damage, fire rate, move speed, max health).
- A doppelgänger's skill snapshot = **whatever permanent skills the player had unlocked by the end of the round that doppelgänger was recorded in**. Later-round doppelgängers are naturally stronger, mirroring your own growth.
- Temp skills (from drops) are player-only, timed, and never copied into a recording.

4.9 Round Progression & Climax

- `TOTAL_ROUNDS_BEFORE_CLIMAX` (default 9). Round 10 = Climax Arena (see Section 15).
-

5. Godot Project Structure

```
res://
├─ autoload/
│   ├─ game_manager.gd          (Autoload: GameManager)
│   └─ events.gd                (Autoload: Events – global signal bus)
├─ resources/
│   ├─ round_recording.gd       (class_name RoundRecording)
│   └─ skill_set.gd             (class_name SkillSet)
├─ scenes/
│   ├─ entities/
│   │   ├─ player.tscn / player.gd
│   │   ├─ doppelganger.tscn / doppelganger.gd
│   │   ├─ enemy.tscn / enemy.gd
│   │   ├─ bullet.tscn / bullet.gd
│   │   ├─ drop.tscn / drop.gd
│   │   ├─ bush.tscn / bush.gd
│   │   ├─ hazard_pit.tscn
│   │   ├─ hazard_spikes.tscn
│   │   ├─ hazard_fire.tscn      (all share hazard.gd)
│   │   └─ clone_control_pickup.tscn
│   ├─ rounds/
│   │   ├─ round_arena.tscn / round_arena.gd
│   │   └─ climax_arena.tscn / climax_arena.gd
│   └─ ui/
│       ├─ hud.tscn / hud.gd
│       ├─ round_transition.tscn
│       ├─ main_menu.tscn
│       └─ victory_screen.tscn
├─ assets/
│   ├─ sprites/
│   ├─ tiles/
│   ├─ sfx/
│   └─ music/
└─ project.godot
```

6. Scene Trees

Player (CharacterBody2D) └─ CollisionShape2D └─ AnimatedSprite2D └─ Gun (Marker2D) └─ RecordingComponent (Node)	Doppelganger (CharacterBody2D) └─ CollisionShape2D └─ AnimatedSprite2D └─ AlertIcon (Sprite2D, hidden default) └─ NavigationAgent2D (optional/stretch)
Enemy (CharacterBody2D) └─ CollisionShape2D └─ AnimatedSprite2D	Hazard (Area2D) └─ CollisionShape2D └─ AnimatedSprite2D / Sprite2D
Bush (Area2D) └─ CollisionShape2D └─ Sprite2D	Drop (Area2D) └─ CollisionShape2D └─ AnimatedSprite2D (bob/spin)
Bullet (Area2D) └─ CollisionShape2D └─ Sprite2D	RoundArena (Node2D) └─ TileMap └─ Hazards (Node2D) └─ Bushes (Node2D) └─ DoppelSpawns (Node2D > Marker2D children) └─ EnemySpawns (Node2D > Marker2D children) └─ Player (instance) └─ CameraRig (Camera2D) └─ HUD (CanvasLayer)

7. Data Resources

res://resources/round_recording.gd

```
extends Resource
class_name RoundRecording

@export var round_number: int = 0
@export var positions: PackedVector2Array = PackedVector2Array()
@export var rotations: PackedFloat32Array = PackedFloat32Array()
@export var actions: Array[Dictionary] = []          # e.g. {frame:int, type:"shoot",
dir:float}
@export var death_hazard_id: String = ""             # "" if the round was completed, not
died
@export var death_frame: int = -1
@export var skills_snapshot: SkillSet
```

Using `PackedVector2Array` / `PackedFloat32Array` instead of an array of `Transform2D` keeps memory light — a 3-minute round at 60 fps is ~10,800 entries per array, which is nothing for Godot, but packed arrays are still the cheaper choice and idiomatic Godot 4.

res://resources/skill_set.gd

```
extends Resource
class_name SkillSet

@export var damage_level: int = 0
@export var fire_rate_level: int = 0
@export var move_speed_level: int = 0
@export var max_health_level: int = 0
@export var has_clone_control: bool = false

func duplicate_skills() -> SkillSet:
    return self.duplicate(true)
```

8. Core Scripts

res://autoload/events.gd (Autoload: Events)

```
extends Node
# Global signal bus – keeps entities from needing direct references to each other.

signal doppelganger_killed(doppelganger)
signal enemy_killed(enemy)
signal skill_unlocked(skill_name)
signal player_damaged(amount)
```

res://autoload/game_manager.gd (Autoload: GameManager)

```
extends Node

signal round_started(round_number: int, doppelganger_count: int)
signal round_ended(round_number: int, player_died: bool)
signal game_won
signal game_over

const TOTAL_ROUNDS_BEFORE_CLIMAX := 9    # rounds 1..9 are normal, round 10 = climax

var current_round: int = 1
var recordings: Array[RoundRecording] = []
var player_skills: SkillSet = SkillSet.new()
var player_currency: int = 0

var xp: int = 0
var xp_to_next_level: int = 10

func _ready() -> void:
    randomize()

func start_new_run() -> void:
    current_round = 1
    recordings.clear()
    player_skills = SkillSet.new()
    player_currency = 0
    xp = 0
    xp_to_next_level = 10
    get_tree().change_scene_to_file("res://scenes/rounds/round_arena.tscn")

func doppelganger_count_for_round(round_number: int) -> int:
    return max(0, round_number - 1)

func finish_round(recording: RoundRecording, player_died: bool) -> void:
    recording.round_number = current_round
    recording.skills_snapshot = player_skills.duplicate_skills()
    recordings.append(recording)
    round_ended.emit(current_round, player_died)

    if current_round >= TOTAL_ROUNDS_BEFORE_CLIMAX:
        get_tree().change_scene_to_file("res://scenes/rounds/climax_arena.tscn")
    else:
        current_round += 1
        get_tree().change_scene_to_file("res://scenes/rounds/round_arena.tscn")

func add_xp(amount: int) -> void:
    xp += amount
    if xp >= xp_to_next_level:
        xp -= xp_to_next_level
        xp_to_next_level = int(xp_to_next_level * 1.4)
        _unlock_random_skill_tier()

func _unlock_random_skill_tier() -> void:
```

```
var choices := ["damage_level", "fire_rate_level", "move_speed_level",  
"max_health_level"]  
var pick: String = choices[randi() % choices.size()]  
player_skills.set(pick, player_skills.get(pick) + 1)  
Events.skill_unlocked.emit(pick)
```

res://scenes/entities/player.gd

```
extends CharacterBody2D
class_name Player

signal died
signal round_completed
signal health_changed(current: int, max: int)

const SPEED := 160.0
const MAX_HEALTH := 3

@onready var recorder: RecordingComponent = $RecordingComponent
@onready var sprite: AnimatedSprite2D = $AnimatedSprite2D
@onready var gun: Marker2D = $Gun

var health: int
var is_possessing_clone: bool = false
var held_clone_control_item: bool = false
var possessed_doppelganger: Doppelganger = null

func _ready() -> void:
    add_to_group("player")
    health = MAX_HEALTH + GameManager.player_skills.max_health_level
    health_changed.emit(health, MAX_HEALTH)

func _physics_process(_delta: float) -> void:
    if is_possessing_clone and possessed_doppelganger:
        _handle_possessed_input()
        return

    var input_dir := Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = input_dir * (SPEED + GameManager.player_skills.move_speed_level * 15)
    move_and_slide()

    sprite.play("run" if input_dir.length() > 0 else "idle")
    if input_dir.length() > 0:
        gun.rotation = input_dir.angle()

    if Input.is_action_just_pressed("shoot"):
        _shoot()
    if Input.is_action_just_pressed("use_clone_control") and held_clone_control_item:
        _activate_clone_control()

func _shoot() -> void:
    recorder.log_action("shoot", {"dir": gun.rotation})
    var bullet := preload("res://scenes/entities/bullet.tscn").instantiate()
    bullet.global_position = gun.global_position
    bullet.direction = Vector2.RIGHT.rotated(gun.rotation)
    bullet.damage = 1 + GameManager.player_skills.damage_level
    bullet.set_meta("source", "player")
    get_tree().current_scene.add_child(bullet)

func take_hit(amount: int = 1) -> void:
```

```

    health -= amount
    health_changed.emit(health, MAX_HEALTH)
    if health <= 0:
        die("")

func heal(amount: int = 1) -> void:
    health = min(health + amount, MAX_HEALTH +
GameManager.player_skills.max_health_level)
    health_changed.emit(health, MAX_HEALTH)

func die(hazard_id: String) -> void:
    recorder.mark_death(hazard_id)
    died.emit()
    GameManager.finish_round(recorder.recording, true)

func complete_round() -> void:
    round_completed.emit()
    GameManager.finish_round(recorder.recording, false)

func _on_hazard_area_entered(hazard: Hazard) -> void:
    die(hazard.hazard_id)

func _activate_clone_control() -> void:
    var target := _find_nearest_doppelganger()
    if target:
        is_possessing_clone = true
        held_clone_control_item = false
        possessed_doppelganger = target
        target.get_possessed(self)
        # hand the camera over – see CameraRig in Section 6
        get_tree().current_scene.get_node("CameraRig").follow_target = target

func release_clone_control() -> void:
    if possessed_doppelganger:
        possessed_doppelganger.release_possession()
    is_possessing_clone = false
    possessed_doppelganger = null
    get_tree().current_scene.get_node("CameraRig").follow_target = self

func on_possessed_clone_destroyed() -> void:
    is_possessing_clone = false
    possessed_doppelganger = null
    get_tree().current_scene.get_node("CameraRig").follow_target = self
    take_hit(1) # shock damage for losing the clone while inside it

func _handle_possessed_input() -> void:
    var input_dir := Input.get_vector("move_left", "move_right", "move_up", "move_down")
    possessed_doppelganger.possessed_move(input_dir)

    if Input.is_action_just_pressed("sabotage"):
        var target := _find_nearest_doppelganger_excluding(possessed_doppelganger)
        if target and
possessed_doppelganger.global_position.distance_to(target.global_position) < 60:
            target.sabotage_from(possessed_doppelganger)

```

```

        _compromise_current_hideout()

    if Input.is_action_just_pressed("release_control"):
        release_clone_control()

func _compromise_current_hideout() -> void:
    for bush in get_tree().get_nodes_in_group("bushes"):
        if bush.hides(global_position):
            bush.compromise()

func _find_nearest_doppelganger() -> Doppelganger:
    var best: Doppelganger = null
    var best_dist := INF
    for d in get_tree().get_nodes_in_group("doppelgangers"):
        var dist := global_position.distance_to(d.global_position)
        if dist < best_dist and dist < 250.0:
            best = d
            best_dist = dist
    return best

func _find_nearest_doppelganger_excluding(exclude: Doppelganger) -> Doppelganger:
    var best: Doppelganger = null
    var best_dist := INF
    for d in get_tree().get_nodes_in_group("doppelgangers"):
        if d == exclude:
            continue
        var dist := exclude.global_position.distance_to(d.global_position)
        if dist < best_dist:
            best = d
            best_dist = dist
    return best

```

res://scenes/entities/recording_component.gd

```
extends Node
class_name RecordingComponent

var recording: RoundRecording
var _frame_count: int = 0

func _ready() -> void:
    recording = RoundRecording.new()

func _physics_process(_delta: float) -> void:
    var body := get_parent() as Node2D
    recording.positions.append(body.global_position)
    recording.rotations.append(body.rotation)
    _frame_count += 1

func log_action(type: String, extra: Dictionary = {}) -> void:
    var entry := {"frame": _frame_count, "type": type}
    entry.merge(extra)
    recording.actions.append(entry)

func mark_death(hazard_id: String) -> void:
    recording.death_hazard_id = hazard_id
    recording.death_frame = _frame_count
```

res://scenes/entities/doppelganger.gd

```
extends CharacterBody2D
class_name Doppelganger

enum State { REPLAY, COMBAT, DODGE, COLLECT, ALERT_SEARCH, POSSESSED }

@export var detection_radius: float = 140.0

var state: State = State.REPLAY
var recording: RoundRecording
var skills: SkillSet
var is_special: bool = false
var health: int = 2
var controller: Player = null

var _frame_index: int = 0
var _fire_cooldown: float = 0.0
var _target_drop: Node = null

@onready var sprite: AnimatedSprite2D = $AnimatedSprite2D
@onready var alert_icon: Sprite2D = $AlertIcon

func setup(rec: RoundRecording, upgraded_special: bool = false) -> void:
    recording = rec
    skills = rec.skills_snapshot
    is_special = upgraded_special
    if is_special:
        skills = SkillSet.new()
        skills.damage_level = 3
        skills.fire_rate_level = 3
        skills.move_speed_level = 2
        skills.max_health_level = 3
        health = 5
        sprite.modulate = Color(1.0, 0.85, 0.2) # gold tint for the elite
    else:
        health = 2 + skills.max_health_level
        sprite.modulate = Color(0.8, 0.2, 0.3) # standard doppelganger tint

func _physics_process(delta: float) -> void:
    _fire_cooldown = max(0.0, _fire_cooldown - delta)
    alert_icon.visible = state in [State.COMBAT, State.ALERT_SEARCH]

    match state:
        State.REPLAY:
            _do_replay()
            _check_player_detection()
            _check_nearby_drops()
        State.COMBAT:
            _do_combat()
        State.DODGE:
            _do_dodge()
        State.COLLECT:
            _do_collect()
```

```

        State.ALERT_SEARCH:
            _do_search(delta)
        State.POSSESSED:
            pass # driven externally via possessed_move()

func _do_replay() -> void:
    if _frame_index >= recording.positions.size():
        state = State.ALERT_SEARCH
        return
    var target_pos: Vector2 = recording.positions[_frame_index]
    global_position = global_position.move_toward(target_pos, 4.0)
    _frame_index += 1
    _replay_actions_for_frame(_frame_index)

func _replay_actions_for_frame(frame: int) -> void:
    for action in recording.actions:
        if action.get("frame") == frame and action.get("type") == "shoot":
            _fire_at(action.get("dir", 0.0))

func _check_player_detection() -> void:
    var player := get_tree().get_first_node_in_group("player") as Player
    if player and not _player_is_hidden(player):
        if global_position.distance_to(player.global_position) < detection_radius:
            state = State.COMBAT

func _player_is_hidden(player: Player) -> bool:
    if not player.is_posessing_clone:
        return false
    for bush in get_tree().get_nodes_in_group("bushes"):
        if bush.hides(player.global_position):
            return true
    return false

func _do_combat() -> void:
    var player := get_tree().get_first_node_in_group("player") as Player
    if not player or _player_is_hidden(player):
        state = State.REPLAY
        return
    var to_player := player.global_position - global_position
    if to_player.length() > detection_radius * 1.5:
        state = State.REPLAY
        return
    var dir := _steer_away_from_known_hazard(to_player.normalized())
    velocity = dir * (100 + skills.move_speed_level * 15)
    move_and_slide()
    if to_player.length() < 220 and _fire_cooldown <= 0.0:
        _fire_at(to_player.angle())

func _steer_away_from_known_hazard(dir: Vector2) -> Vector2:
    if recording.death_hazard_id == "":
        return dir
    var hazard := get_tree().get_first_node_in_group(recording.death_hazard_id)
    if hazard and global_position.distance_to(hazard.global_position) < 80:
        var away := (global_position - hazard.global_position).normalized()

```

```

        return (dir + away).normalized()
    return dir

func _fire_at(angle: float) -> void:
    _fire_cooldown = 1.2 - skills.fire_rate_level * 0.15
    var bullet := preload("res://scenes/entities/bullet.tscn").instantiate()
    bullet.global_position = global_position
    bullet.direction = Vector2.RIGHT.rotated(angle)
    bullet.damage = 1 + skills.damage_level
    bullet.set_meta("source", "doppelganger")
    get_tree().current_scene.add_child(bullet)

func _do_dodge() -> void:
    # Simple version: strafe perpendicular to the last known threat direction.
    # Expand later with real bullet-proximity checks if time allows.
    state = State.COMBAT

func _check_nearby_drops() -> void:
    for drop in get_tree().get_nodes_in_group("drops"):
        if global_position.distance_to(drop.global_position) < 100:
            _target_drop = drop
            state = State.COLLECT
            break

func _do_collect() -> void:
    if not is_instance_valid(_target_drop):
        state = State.REPLAY
        return
    global_position = global_position.move_toward(_target_drop.global_position, 3.0)
    if global_position.distance_to(_target_drop.global_position) < 10:
        _target_drop.collect(self)
        _target_drop = null
        state = State.REPLAY

func _do_search(delta: float) -> void:
    var player := get_tree().get_first_node_in_group("player") as Player
    if player and not _player_is_hidden(player):
        if global_position.distance_to(player.global_position) < detection_radius * 1.3:
            state = State.COMBAT
            return
    # naive wander – replace with NavigationAgent2D pathing if time allows
    velocity = velocity.rotated(randf_range(-0.3, 0.3)).normalized() * 70
    move_and_slide()

func take_hit(amount: int, from_player: bool = true) -> void:
    health -= amount
    if health <= 0:
        _die(from_player)

func heal(amount: int = 1) -> void:
    health += amount

func _die(killed_by_player: bool) -> void:
    if state == State.POSSESSED and controller:

```

```

        controller.on_possessed_clone_destroyed()
    if killed_by_player:
        GameManager.player_currency += 5
        GameManager.add_xp(3)
        Events.doppelganger_killed.emit(self)
    if is_special:
        var pickup :=
preload("res://scenes/entities/clone_control_pickup.tscn").instantiate()
        pickup.global_position = global_position
        get_tree().current_scene.add_child(pickup)
        queue_free()

func get_possessed(player: Player) -> void:
    state = State.POSSESSED
    controller = player

func release_possession() -> void:
    state = State.ALERT_SEARCH
    controller = null

func possessed_move(dir: Vector2) -> void:
    velocity = dir * (140 + skills.move_speed_level * 15)
    move_and_slide()

func sabotage_from(_saboteur: Doppelganger) -> void:
    _die(true)
    _broadcast_alert()

func _broadcast_alert() -> void:
    for d in get_tree().get_nodes_in_group("doppelgangers"):
        if d != self and is_instance_valid(d):
            d.state = State.ALERT_SEARCH

```

res://scenes/entities/enemy.gd

```
extends CharacterBody2D
class_name Enemy

@export var aggro_radius: float = 120.0
@export var speed: float = 70.0

var health: int = 2

func _ready() -> void:
    add_to_group("enemies")

func _physics_process(_delta: float) -> void:
    var player := get_tree().get_first_node_in_group("player") as Player
    if player and global_position.distance_to(player.global_position) < aggro_radius:
        velocity = (player.global_position - global_position).normalized() * speed
        move_and_slide()

func _on_body_entered(body: Node) -> void:
    if body is Player:
        body.take_hit(1)
    elif body is Doppelganger:
        body.take_hit(1, false)

func take_hit(amount: int) -> void:
    health -= amount
    if health <= 0:
        Events.enemy_killed.emit(self)
        GameManager.add_xp(1)
        _drop_loot()
        queue_free()

func _drop_loot() -> void:
    if randf() < 0.5:
        var drop := preload("res://scenes/entities/drop.tscn").instantiate()
        drop.kind = "coin" if randf() < 0.7 else "health"
        drop.global_position = global_position
        get_tree().current_scene.add_child(drop)
```

res://scenes/entities/drop.gd

```
extends Area2D
class_name Drop

@export var kind: String = "coin"    # "coin" | "health"

func _ready() -> void:
    add_to_group("drops")

func _on_body_entered(body: Node) -> void:
    # Both the player AND doppelgangers can trigger this – see Section 4.7.
    if body is Player or body is Doppelganger:
        collect(body)

func collect(who: Node) -> void:
    match kind:
        "coin":
            if who is Player:
                GameManager.player_currency += 1
            # a doppelganger picking up a coin just denies it to the player
        "health":
            if who.has_method("heal"):
                who.heal(1)
    queue_free()
```

res://scenes/entities/hazard.gd

```
extends Area2D
class_name Hazard

@export var hazard_id: String = "hazard_pit_01"    # unique per placed instance

func _ready() -> void:
    add_to_group(hazard_id)    # lets a doppelganger recall "the hazard that killed me"
    add_to_group("hazards")

func _on_body_entered(body: Node) -> void:
    if body is Player:
        body.die(hazard_id)
    elif body is Doppelganger:
        body.take_hit(999)
```

res://scenes/entities/bush.gd

```
extends Area2D
class_name Bush

var compromised: bool = false

func _ready() -> void:
    add_to_group("bushes")

func hides(pos: Vector2) -> bool:
    return not compromised and global_position.distance_to(pos) < 40

func compromise() -> void:
    compromised = true
    modulate = Color(1, 0.6, 0.6)    # visual tell that this hideout is blown
```

res://scenes/entities/bullet.gd

```
extends Area2D
class_name Bullet

@export var speed: float = 400.0
var direction: Vector2 = Vector2.RIGHT
var damage: int = 1
var _lifetime: float = 2.0

func _physics_process(delta: float) -> void:
    position += direction * speed * delta
    _lifetime -= delta
    if _lifetime <= 0:
        queue_free()

func _on_body_entered(body: Node) -> void:
    var source: String = get_meta("source", "")
    if source == "player" and (body is Enemy or body is Doppelganger):
        body.take_hit(damage)
        queue_free()
    elif source == "doppelganger" and body is Player:
        body.take_hit(damage)
        queue_free()
    elif body is Hazard or body.is_in_group("walls"):
        queue_free()
```

res://scenes/rounds/round_arena.gd

```
extends Node2D
class_name RoundArena

var _special_timer: Timer

func _ready() -> void:
    GameManager.round_started.emit(GameManager.current_round,
    GameManager.recordings.size())
    _spawn_doppelgangers()
    _spawn_enemies()
    _start_special_doppel_timer()

func _spawn_doppelgangers() -> void:
    var doppel_scene := preload("res://scenes/entities/doppelganger.tscn")
    var spawn_points := $DoppelSpawns.get_children()
    for i in GameManager.recordings.size():
        var rec: RoundRecording = GameManager.recordings[i]
        var d := doppel_scene.instantiate()
        d.add_to_group("doppelgangers")
        var spawn: Node2D = spawn_points[i % spawn_points.size()]
        d.global_position = spawn.global_position
        add_child(d)
        d.setup(rec)

func _spawn_enemies() -> void:
    var enemy_scene := preload("res://scenes/entities/enemy.tscn")
    var spawn_points := $EnemySpawns.get_children()
    var count := 3 + GameManager.current_round
    for i in count:
        var e := enemy_scene.instantiate()
        var spawn: Node2D = spawn_points[randi() % spawn_points.size()]
        e.global_position = spawn.global_position + Vector2(randf_range(-20, 20),
        randf_range(-20, 20))
        add_child(e)

func _start_special_doppel_timer() -> void:
    _special_timer = Timer.new()
    _special_timer.wait_time = 120.0
    _special_timer.autostart = true
    _special_timer.timeout.connect(_upgrade_random_doppelganger)
    add_child(_special_timer)

func _upgrade_random_doppelganger() -> void:
    var doppels := get_tree().get_nodes_in_group("doppelgangers")
    if doppels.is_empty():
        return
    var chosen: Doppelganger = doppels[randi() % doppels.size()]
    chosen.setup(chosen.recording, true)
```

res://scenes/rounds/climax_arena.gd

```
extends Node2D
class_name ClimaxArena

func _ready() -> void:
    GameManager.round_started.emit(GameManager.current_round,
    GameManager.recordings.size())
    Events.doppelganger_killed.connect(_check_victory)
    _spawn_all_doppelgangers_hostile()

func _spawn_all_doppelgangers_hostile() -> void:
    var doppel_scene := preload("res://scenes/entities/doppelganger.tscn")
    var spawn_points := $DoppelSpawns.get_children()
    for i in GameManager.recordings.size():
        var rec: RoundRecording = GameManager.recordings[i]
        var d := doppel_scene.instantiate()
        d.add_to_group("doppelgangers")
        var spawn: Node2D = spawn_points[i % spawn_points.size()]
        d.global_position = spawn.global_position
        add_child(d)
        d.setup(rec)
        d.state = Doppelganger.State.COMBAT # skip the replay phase, go straight
        hostile

func _check_victory(_d) -> void:
    await get_tree().process_frame
    if get_tree().get_nodes_in_group("doppelgangers").is_empty():
        GameManager.game_won.emit()
        get_tree().change_scene_to_file("res://scenes/ui/victory_screen.tscn")
```

CameraRig (attach directly to a Camera2D node in each arena scene)

```
extends Camera2D
class_name CameraRig

var follow_target: Node2D

func _process(_delta: float) -> void:
    if is_instance_valid(follow_target):
        global_position = global_position.lerp(follow_target.global_position, 0.15)
```

Set `follow_target = player` in `RoundArena._ready()`; `Player._activate_clone_control()` / `release_clone_control()` already swap it for you.

9. Input Map (Project Settings → Input Map)

Action	Suggested key(s)
move_left	A / Left Arrow
move_right	D / Right Arrow
move_up	W / Up Arrow
move_down	S / Down Arrow
shoot	Left Mouse Button / Space
use_clone_control	E
sabotage	F (only meaningful while possessed)
release_control	Q

10. Sprite & Asset List

Characters

Asset	States needed	Notes
Player	idle, run, shoot, hit-flash, death	32×32 or 48×48, top-down/¾ view, keep it simple and readable
Doppelgänger	same set as player	Same silhouette as player but recolored (e.g. dark red tint) so they're instantly readable as "not you" — plus an "alert"  overlay icon
Special/Elite Doppelgänger	same set, gold/glowing tint	Slightly larger scale or added outline/aura
Player husk (idle unpossessed body)	reuse Player idle, dimmed/greyed	Visual cue it's vulnerable
Enemy	idle, chase/move, death	Distinct silhouette from player/doppelganger — 1 type is enough for MVP

Environment

Asset	Notes
Floor tile	Tileable, 32×32 or 64×64
Wall/border tile	Arena boundary
Hazard: Pit	Static dark hole decal
Hazard: Spikes	2 frames min (retracted/extended)
Hazard: Fire	4–6 frame looping flame
Bush	2 states: normal (hiding) vs. compromised (trampled/reddish tint)

Pickups / Projectiles

Asset	Notes
Coin	small spin/bob idle loop
Health pack	cross or potion icon
Clone-control pickup	glowing mask/orb — thematically ties to "doppelganger," pulsing glow anim
Temp-skill drop(s)	2–3 icons: lightning (fire rate), boots (speed), shield (defense)
Bullet	small circle/shard + optional muzzle flash frame

VFX

- Hit spark / impact flash
- Death poof (enemies/doppelgangers)
- Alert  icon pop
- Possession swirl (enter/exit clone control)
- Sabotage burst
- Elite aura/glow

11. Audio List

- **SFX:** footsteps, gunshot, hit/impact, enemy death, doppelganger death, pickup collect, clone-control activate/deactivate, sabotage trigger, alert stinger, round transition.
 - **Music:** one tense-but-calm loop for normal rounds, one intensified loop for the climax.
-

12. UI / HUD List

- Health pips/hearts
 - Round counter ("Round 4 / 9")
 - Doppelgänger counter
 - Currency counter
 - Clone-control item slot (empty/held/ready states)
 - Skill-unlock popup (fires on `Events.skill_unlocked`)
 - Round-transition screen (fade + "Round Complete" or death flavor text)
 - Climax intro screen
 - Victory / defeat screens
-

13. Build Order (Jam Milestones)

1. **Setup** — project, folders, Input Map, git repo.
2. **Core movement & shooting** — Player controller + Bullet, empty test arena.
3. **Hazards & round loop skeleton** — Hazard node, a round-complete trigger, GameManager round transitions (even with 0 doppelgangers).
4. **Recording system** — RecordingComponent + RoundRecording; sanity check by printing array sizes.
5. **Doppelgänger replay only** — spawn one ghost next round that just walks its old path, no combat yet. Confirm the core "time-loop" feeling works.
6. **Doppelgänger combat AI** — detection → combat → hazard-avoidance.
7. **Enemies** — chaser + contact damage + loot drop.
8. **Economy & skills** — coins, health packs, XP → permanent skill unlocks, temp-skill drops.
9. **Clone control & possession** — pickup item, possess/release, camera hand-off, vulnerable husk.
10. **Sabotage, alert & bushes** — sabotage action, alert broadcast/search state, bush hide + compromise.
11. **Elite doppelgänger loop** — timed upgrade, elite stats, clone-control pickup drop.
12. **Progression & climax** — round counter to 8–10, climax arena, win/lose screens.
13. **UI/HUD & juice** — all HUD elements, VFX, SFX, animation polish.
14. **Playtest & balance pass** — tune detection radii, damage, spawn counts, elite timer.

Phases 9–10 (possession + sabotage) are the highest-risk/highest-complexity features — budget accordingly and don't start them until 1–8 are solid.

14. Scope: MVP vs. Nice-to-Have vs. Stretch

MVP (must have a playable start-to-finish loop):

- Movement + shoot

- At least 1 hazard type, instant death
- Round loop with a defined end (even "round 5 = win")
- Doppelgänger replay (movement only counts as a great minimal version)
- 1 enemy type (chaser + contact damage)
- Round counter

Nice-to-have:

- Doppelgänger combat AI (detect/attack/dodge/avoid-hazard)
- Loot drops + currency
- Permanent skill unlocks via XP
- Basic HUD (health/round/currency)

Stretch:

- Clone-control possession
- Sabotage + alert propagation
- Bushes/hiding
- Elite doppelgänger loop
- Temp-skill drops
- Full climax encounter
-  Doppelgängers collecting enemy drops (depends on both AI states and the drop system being done — build this last, it's a small addition once `Doppelganger.COLLECT` and `Drop.gd` already exist)

15. Final Climax Options

Option A — "The Horde" (cheapest to build): all 8–9 accumulated doppelgängers spawn at once, skip the replay phase, go straight to `COMBAT / ALERT_SEARCH`. Reuses 100% of existing systems — no new code beyond `climax_arena.gd` above.

Option B — "Alpha Doppelgänger": one boss-tier doppelgänger combining the *best* stats from every recorded round (max damage/fire-rate/speed/health across all snapshots), plus a handful of normal doppelgängers as adds. More climactic, but needs a bespoke boss script and balancing pass.

Option C — Hybrid: Alpha boss + reduced horde. Best drama-to-effort ratio if you have time after Option A is working — start with A, upgrade to C if the jam clock allows.

16. Playtesting & Balance Checklist

- ☐ Detection radius feels fair (not omniscient, not blind)
- ☐ 3-hit death doesn't feel too punishing/too forgiving against enemy contact damage

- ☐ Elite timer (2 min) — test if it should scale down in later rounds for pacing
 - ☐ Doppelgänger count in round 8–9 doesn't overwhelm the arena visually/performance-wise
 - ☐ Sabotage risk/reward — does alert search actually threaten the hidden body, or is it toothless?
 - ☐ Bush compromise is visually obvious enough that players understand why they got caught
 - ☐ XP curve — permanent skills shouldn't trivialize mid-game, but should feel earned
-

17. Quick-Start Checklist for Jam Night

- ☐ Create project, folders, git repo
 - ☐ Input Map set up (Section 9)
 - ☐ Player scene walking + shooting in a blank room
 - ☐ One hazard type killing the player
 - ☐ GameManager autoload wired, round loop confirmed via `print()`
 - ☐ First doppelgänger replaying a recorded path
 - ☐ First enemy chasing and dealing contact damage
 - ☐ HUD showing health + round number
 - ☐ Everything else from Section 13, in order, as time allows
-

18. Known Pitfalls & Mitigations (Read Before You Build)

Things that will actually bite you mid-jam, with fixes. Skim this before you start Section 4.4/4.5 (doppelgänger + possession) — that's where most of these live.

18.1 Spawn-related bugs

- **Instant spawn-aggro.** `_check_player_detection()` runs on frame 1 of `REPLAY` with no grace period, and there's nothing stopping a doppelgänger spawn marker from landing inside the player's detection radius — result: ghosts shoot you before you've even moved. Fix: spawn grace timer + minimum spawn distance from the player's spawn point.

```
# doppelganger.gd
var _spawn_grace: float = 1.5

func _physics_process(delta: float) -> void:
    _spawn_grace = max(0.0, _spawn_grace - delta)
    match state:
        State.REPLAY:
            _do_replay()
            if _spawn_grace <= 0.0:
                _check_player_detection()
                _check_nearby_drops()
```

```
# round_arena.gd
const MIN_SPAWN_DISTANCE_FROM_PLAYER := 200.0

func _spawn_doppelgangers() -> void:
    var player_spawn: Vector2 = $PlayerSpawn.global_position
    var valid_points := $DoppelSpawns.get_children().filter(
        func(p): return p.global_position.distance_to(player_spawn) >
MIN_SPAWN_DISTANCE_FROM_PLAYER
    )
    for i in GameManager.recordings.size():
        var d := preload("res://scenes/entities/doppelganger.tscn").instantiate()
        d.add_to_group("doppelgangers")
        d.global_position = valid_points[i % valid_points.size()].global_position
        add_child(d)
        d.setup(GameManager.recordings[i])
```

- **Doppelgängers stacking on top of each other.** Spawn point count needs to be $\geq \text{TOTAL_ROUNDS_BEFORE_CLIMAX} - 1$ (8 by default). Fewer than that and the `i % spawn_points.size()` wraparound puts multiple ghosts on the same marker — they'll overlap or violently collide-push apart.
- **Spawning inside a hazard.** A doppelgänger spawned inside a pit/spike zone dies instantly via `Hazard._on_body_entered`. Keep spawn markers clear of hazard radii, or extend spawn-grace immunity to hazards too.
- **Scene-tree ready-order race.** If a doppelgänger's `_physics_process` runs before `Player._ready()` has called `add_to_group("player")`, `get_first_node_in_group("player")` returns null → crash on frame one. Spawn the Player node before doppelgängers in the tree, or null-check defensively.

18.2 Replay-fidelity bugs

- **Position drift.** `move_toward(target_pos, 4.0)` is a fixed pixel-per-frame step. If the player ever moves faster than that between two recorded frames (a speed skill, a dash, a lag spike), the replay falls behind and never catches up. Safer: snap directly (`global_position = target_pos`) rather than stepping toward it.
- **Ghosts walk back into the trap that killed them.** Hazard-avoidance is currently only coded in `COMBAT`, not `REPLAY`. During plain replay, a doppelgänger will re-enact its own death. Decide if that's intentional flavor or a bug to fix — if the notes' "avoids traps it died to" should hold during replay too, add the same steering check inside `_do_replay()`.
- **Stale recordings if the arena changes between rounds.** Recorded positions are absolute world coordinates. If hazard/enemy layout is randomized per round, a ghost will path through where a hazard *used to be*, or walk into a *new* one it has no memory of. Keep the arena layout identical for the whole run, or switch to relative-position recording.
- **Idle padding.** If `RecordingComponent` keeps running during round-transition fades/pause menus, you bake in frozen frames — next doppelgänger stands still for a beat before moving. Explicitly pause recording during transitions.

- **Possessed-clone actions aren't recorded anywhere.** While piloting a doppelgänger, the real body's `RecordingComponent` just logs it standing still. Nothing captures what the possessed clone did. Fine as a decision (future ghosts show your real body idling in a bush) — just make it a deliberate one, not a surprise.

18.3 AI state-machine gotchas

- **Old, quick-death doppelgängers run out of script early.** A doppelgänger born from an 8-second death only has 8 seconds of replay. Once `_frame_index` exceeds it, it flips to `ALERT_SEARCH` and starts hunting — often well before the round ends. A delayed version of the spawn-aggro problem; worth tuning in playtesting.
- **"Blending in" currently works for free.** Nothing distinguishes a possessed doppelgänger from a normal one to its peers — they only ever attack each other via deliberate sabotage. So there's no real detection risk while infiltrating unless you add a suspicion mechanic on top.
- **Elite-timer can re-roll badly.** The 2-minute upgrade can pick a doppelgänger that's already special (wasted), or the one you're *currently possessing* (`setup()` resets its HP — a free-heal exploit), or spawn a second elite if the first isn't killed in time. Filter the candidate pool:

```
func _upgrade_random_doppelganger() -> void:
    var candidates := get_tree().get_nodes_in_group("doppelgangers").filter(
        func(d): return not d.is_special and d.state != Doppelganger.State.POSSESSED
    )
    if candidates.is_empty():
        return
    var chosen: Doppelganger = candidates[randi() % candidates.size()]
    chosen.setup(chosen.recording, true)
```

18.4 Classic Godot reference bugs (will hit you regardless of this system)

- **"Trying to call a function on a previously freed instance."** Cached references (`possessed_doppelganger`, `_target_drop`, the camera's `follow_target`) must be nulled the instant that node is freed. Guard with `is_instance_valid()` before touching anything you didn't spawn this frame.
- **Silent resource-sharing bug.** `recording.skills_snapshot` must be `player_skills.duplicate_skills()`, never a direct reference — Resources are reference types in Godot. If that `.duplicate()` call ever gets "simplified" away, every past doppelgänger silently keeps leveling up alongside you for the rest of the run, with no error to warn you.
- **Hand-typed hazard_id typos.** Since it doubles as a Godot group name, a duplicated or mistyped ID means avoidance logic references the wrong hazard, or none at all, and fails silently. Prefer auto-generating IDs (`"hazard_%d" % get_instance_id()`) over typing them in the Inspector.

18.5 Performance

By round 8–9, up to 9 doppelgängers each call `get_first_node_in_group("player")` and scan `get_nodes_in_group("drops") / "hazards"` every physics frame. Fine at jam scale, but cache the

player reference once instead of re-fetching per doppelgänger per frame, and profile before the climax round — especially for a web/HTML5 export.

18.6 Quick Reference

Symptom	Likely cause	Fix
Clones shoot you the instant a round starts	No spawn grace period + detection runs immediately	Spawn-immunity timer + minimum spawn distance from player
Clones stacked/jittering at round start	Doppelgänger count > spawn point count	More spawn markers, or jitter offset
A doppelgänger dies the moment it spawns	Spawned inside a hazard's Area2D	Keep spawn markers clear of hazard zones
Doppelgänger walks straight back into the trap that killed it	Avoidance logic only runs in COMBAT, not REPLAY	Decide intent; add avoidance to <code>_do_replay()</code> if unwanted
Doppelgänger behaves oddly or teleports after arena changes	Hazard/layout changed between rounds — recording is stale	Keep arena layout identical per run
"Trying to call a function on a previously freed instance"	Stale reference to a <code>queue_free()</code> 'd node	<code>is_instance_valid()</code> check / null references immediately on free
Doppelgängers keep getting stronger even after the round they were born in	SkillSet snapshot wasn't duplicated	Always <code>.duplicate()</code> before storing
A hazard's avoidance never triggers, or triggers for the wrong hazard	Duplicate/typo'd <code>hazard_id</code> string	Auto-generate unique IDs instead of hand-typing
Frame rate drops in later rounds (esp. web export)	Per-frame group scans across many entities	Cache player reference; profile before the climax round